

Smart Data Warehouse & Agentic NL2SQL System

Full Technical Project Report

Olist Brazilian E-Commerce | MySQL 8.0 | 5-Agent AI Pipeline

99,441 orders | Star-schema warehouse | ReAct agentic loop | 10 advanced features

Shreya Ghatage

IIT Madras -- Data Science Programme

22f3000165@ds.study.iitm.ac.in

Branch: develop

Commits: 15 (spanning full development arc)

Tests: 54 passing

Date: June 2026

Built with Python 3.11 . MySQL 8.0 . Ollama . Groq . Streamlit . Plotly . FastAPI

Abstract

This report documents the complete end-to-end development of the Smart Data Warehouse NL2SQL System -- an agentic artificial intelligence platform that converts natural-language business questions into executable SQL, runs them against a production-grade MySQL 8.0 star-schema data warehouse, and returns interactive charts alongside LLM-generated business insights.

The project is built on the Olist Brazilian E-Commerce dataset (2016-2018), comprising 99,441 orders across 27 Brazilian states and nine raw CSV files. A production-grade ETL pipeline transforms the raw data into a Kimball-style star schema with one fact table (`fact_sales`) and four dimension tables, incorporating seven derived features including delivery time, delay days, and a binary late-delivery flag.

The AI core is a five-agent sequential pipeline evolved into a ReAct-style agentic loop: Agent 1 parses intent, Agent 2 generates SQL, Agent 3 validates safety and schema correctness, Agent 4 executes against MySQL, and Agent 5 generates business insights. A dedicated Reflection Agent classifies failures, diagnoses root causes, and plans targeted recovery strategies between attempts -- replacing raw error passthrough with structured reasoning.

The user-facing Streamlit application provides four pages (Query, Dashboard, History, Admin), supplemented by ten major feature upgrades: multi-turn conversational memory, Brazil geospatial bubble maps, time-series forecasting, PDF report export, a FastAPI REST endpoint, RFM customer segmentation, confidence scoring, semantic query caching, voice input, and multi-LLM benchmarking.

The project demonstrates a complete data engineering lifecycle -- from raw CSV ingestion through multi-layer cleaning, star-schema warehousing, agentic AI, and a polished product-quality UI -- in approximately 4,000 lines of Python across 30+ modules.

1. Introduction

1.1 Problem Statement

Business users need answers from data warehouses but lack SQL expertise. Traditional BI tools require either pre-built dashboards (inflexible) or direct SQL access (inaccessible). Large Language Models can generate SQL from natural language, but naive single-LLM approaches suffer from hallucination, schema mismatch, and unsafe queries.

The core challenge: build a system that is simultaneously accurate (generates correct SQL for a specific schema), safe (never executes destructive statements), robust (recovers from LLM errors automatically), and accessible (usable by non-technical stakeholders via a polished UI).

1.2 Project Objectives

- Design and populate a production-grade MySQL star-schema warehouse from the Olist E-Commerce dataset.
- Build a multi-agent NL2SQL pipeline with five specialised agents.
- Evolve the pipeline into a genuine agentic loop using the ReAct pattern with failure reflection.
- Deliver a polished multi-page Streamlit UI with real-time charts, AI insights, and query history.
- Expose the pipeline as a documented REST API (FastAPI).
- Add ten advanced features demonstrating data science, ML, and product engineering depth.

1.3 Significance

This project sits at the intersection of three currently active research and engineering areas: Text-to-SQL (NL2SQL), agentic AI systems (multi-step LLM orchestration with self-correction), and analytical data warehousing (Kimball star schema, ETL, performance indexing). The combination produces a system that is both academically grounded and production-realistic.

2. Dataset: Olist Brazilian E-Commerce

2.1 Overview

The Olist dataset is a publicly available real-world e-commerce dataset from Brazil's largest online marketplace. It covers orders placed between September 2016 and October 2018 and is released under a Creative Commons licence. Nine CSV files are provided.

File	Contents	Rows
olist_orders_dataset.csv	Order headers: status, timestamps	99,441
olist_order_items_dataset.csv	Per-item price, freight, seller	112,650
olist_order_payments_dataset.csv	Payment type, instalment, value	103,886
olist_order_reviews_dataset.csv	Customer review scores & text	99,224
olist_customers_dataset.csv	Customer city, state, zip code	99,441
olist_sellers_dataset.csv	Seller city, state, zip code	3,095
olist_products_dataset.csv	Category, dimensions, weight	32,951
olist_geolocation_dataset.csv	Zip -> lat/lon mappings	1,000,163
product_category_name_translation.csv	Portuguese->English labels	71

2.2 Key Metrics (Delivered Orders)

Metric	Value
Total delivered orders	~96,000
Total revenue	R\$ 16.0M+
Unique customers	~96,000
Unique sellers	3,095
Product categories (English)	74
Brazilian states covered	27

Metric	Value
Date range	Sept 2016 - Oct 2018
Average review score	4.07 / 5
Average delivery time	~12 days
Late delivery rate	~7%

2.3 Data Quality Issues

- Typos in column names: product_name_lenght -> product_name_length (corrected in cleaning).
- ~610 products had no Portuguese category name; filled as 'unknown' post-merge.
- Orders with null order_delivered_customer_date produce NaN delivery times; handled with `pd.to_datetime(errors='coerce')`.
- Some orders had multiple payment records (instalment rows); aggregated to primary type and total value at order level.
- Geolocation CSV (1M rows) was not used in the warehouse (redundant with state/city columns); excluded from ETL.

3. System Architecture

3.1 High-Level Design

The system follows a layered architecture with four distinct tiers. Each tier communicates only with its immediate neighbour, ensuring clean separation of concerns.

Tier 4 -- UI	Streamlit Multi-Page UI (4 pages) + FastAPI REST
Tier 3 -- AI	5-Agent NL2SQL Pipeline + Reflection Agent (ReAct loop)
Tier 2 -- SQL	SQL Validation + MySQL Execution Engine
Tier 1 -- Data	MySQL 8.0 Star-Schema Warehouse (5 tables)
Tier 0 -- ETL	ETL Pipeline: 9 Olist CSVs -> Warehouse

3.2 Technology Stack

Layer	Technology	Purpose
LLM (local)	Ollama + Llama 3 / Mistral	Private, zero-cost inference
LLM (cloud)	Groq API + llama3-70b	Fast cloud inference (free tier)
Database	MySQL 8.0	Star-schema warehouse
ORM / SQL	SQLAlchemy 2.0 + PyMySQL	Connection pooling, safe query execution
Data wrangling	Pandas 2.2 + NumPy 1.26	ETL transformations, feature engineering
Web UI	Streamlit 1.35	Multi-page interactive app
REST API	FastAPI + Uvicorn	Programmatic pipeline access
Charts	Plotly 5.22	Interactive visualisations
ML	scikit-learn	Forecasting, RFM clustering
Validation	sqlparse 0.5 + regex	SQL safety checks
Retry	tenacity 3.x	Exponential-backoff DB retries
Logging	loguru 0.7	Structured JSONL query log
PDF export	fpdf2	Branded PDF report generation
Cache	sklearn TF-IDF	Semantic query similarity cache

Layer	Technology	Purpose
Testing	pytest 8.2 + pytest-cov	Unit + integration test suite

4. Database Design: Star Schema

4.1 Design Philosophy

The warehouse follows the Kimball Dimensional Modelling approach: a central fact table recording transactional events, surrounded by denormalised dimension tables. This design optimises for analytical queries (GROUP BY, aggregations, date-range slices) at the expense of write normalisation -- appropriate for a read-heavy analytical workload.

4.2 Star Schema Tables

Fact Table: fact_sales

One row per order-item. Records all measurable business events.

fact_sales DDL

```
CREATE TABLE fact_sales (
  fact_id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  order_id         VARCHAR(36) NOT NULL,
  customer_id      VARCHAR(36),    -- FK -> dim_customers
  product_id       VARCHAR(36),    -- FK -> dim_products
  seller_id        VARCHAR(36),    -- FK -> dim_sellers
  date_key         INT,            -- FK -> dim_time (YYYYMMDD)
  order_status     VARCHAR(20),
  payment_type     VARCHAR(20),
  payment_value    DECIMAL(10,2),
  review_score     TINYINT,
  delivery_time_days DECIMAL(6,2), -- DERIVED feature
  delay_days       DECIMAL(6,2),   -- positive = late
  is_late          TINYINT(1),     -- DERIVED binary flag
  approval_time_hours DECIMAL(8,2) -- DERIVED feature
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Dimension Tables

Table	Purpose	Rows	Key Columns
dim_time	Calendar dimension	~750	year, month, quarter, day_of_week
dim_customers	Customer geography	99,441	customer_state, customer_city
dim_products	Product attributes + EN category	32,951	category_name_english
dim_sellers	Seller geography	3,095	seller_state, seller_city

4.3 Indexing Strategy

MySQL 8.0 enforces ONLY_FULL_GROUP_BY mode. Composite indexes were chosen to support the most common analytical patterns.

Composite indexes

```
CREATE INDEX idx_fact_date_cust ON fact_sales(date_key, customer_id);
CREATE INDEX idx_fact_product ON fact_sales(product_id, payment_value);
CREATE INDEX idx_fact_seller ON fact_sales(seller_id, review_score);
CREATE INDEX idx_time_year_month ON dim_time(year, month);
CREATE INDEX idx_cust_state_city ON dim_customers(customer_state, customer_city);
```

5. ETL Pipeline

5.1 Pipeline Stages

Five sequential stages, each a dedicated Python module under etl/.

Stage 1 -- Ingestion	<code>data_ingestion.py</code>	Reads all 9 Olist CSVs into a dict of DataFrames
Stage 2 -- Cleaning	<code>data_cleaning.py</code>	Dedup, datetime parse, string normalisation, typo fix, cat
Stage 3 -- Integration	<code>data_integration.py</code>	Joins orders, items, payments, reviews into a master ord
Stage 4 -- Feature Engineering	<code>feature_engineering.py</code>	Derives delivery_time_days, delay_days, is_late, approv
Stage 5 -- Loading	<code>data_loading.py</code>	FK-safe TRUNCATE then INSERT into star schema tabl

5.2 Feature Engineering

Four business-critical features derived from timestamp arithmetic:

Derived feature computation (feature_engineering.py)

```
df['delivery_time_days'] = (
    (df['order_delivered_customer_date']
     - df['order_purchase_timestamp'])
    .dt.total_seconds() / 86400
).round(2)

df['delay_days'] = (
    (df['order_delivered_customer_date']
     - df['order_estimated_delivery_date'])
    .dt.total_seconds() / 86400
).round(2)    # positive = late, negative = early

df['is_late'] = (df['delay_days'] > 0).astype(int)
df['approval_time_hours'] = (
    (df['order_approved_at'] - df['order_purchase_timestamp'])
    .dt.total_seconds() / 3600
).round(2)
```

5.3 FK-Safe Re-load Pattern

[i] Critical Bug Fix -- Commit 62397d8

Using `if_exists='replace'` dropped and recreated dimension tables, causing FK constraint violations on the fact table. Fix: disable FK checks, truncate all tables (fact first), re-enable, then re-insert. This makes ETL runs idempotent.

FK-safe truncate pattern

```
conn.execute(text('SET FOREIGN_KEY_CHECKS=0'))
for t in ['fact_sales', 'dim_customers', 'dim_products', 'dim_sellers', 'dim_time']:
    conn.execute(text(f'TRUNCATE TABLE {t}'))
conn.execute(text('SET FOREIGN_KEY_CHECKS=1'))
# then insert dims, then fact
```

6. Multi-Agent NL2SQL Pipeline

6.1 Design Philosophy

Each agent has exactly one responsibility, receives a typed input, and returns a typed output. No agent knows about agents after it. The pipeline orchestrator is the only component that sees the full state. This separation of concerns enables the ReAct agentic loop.

6.2 Agent 1 -- Query Understanding

Input: Raw natural-language question. Output: Structured JSON intent dict. Agent 1 sends the question to the LLM with a zero-shot prompt specifying the exact JSON schema to produce.

Intent JSON schema

```
{
  "intent":    "aggregation | trend | comparison | lookup | ranking",
  "metric":   "revenue | orders | review_score | delivery_time_days",
  "dimension": "customer_state | product_category_name_english | ...",
  "filters":  {"year": 2018, "order_status": "delivered"},
  "limit":    5,
  "sort_order": "DESC",
  "time_grain": "monthly | quarterly | annual | null"
}
```

6.3 Agent 2 -- NL2SQL Generation

Input: Intent JSON + original question + optional reflection feedback. Output: Raw MySQL SELECT statement. The system prompt explicitly enforces MySQL 8.0's ONLY_FULL_GROUP_BY rules with WRONG/CORRECT examples.

ONLY_FULL_GROUP_BY enforcement examples in SQL agent prompt

```
-- WRONG (fails on MySQL 8.0 with Error 1055):
SELECT dt.full_date, SUM(fs.payment_value) AS revenue
FROM fact_sales fs JOIN dim_time dt ON fs.date_key=dt.date_key
GROUP BY YEAR(dt.full_date), MONTH(dt.full_date)

-- CORRECT:
SELECT dt.year, dt.month,
       CONCAT(dt.year, '-', LPAD(dt.month,2,'0')) AS month_label,
       ROUND(SUM(fs.payment_value),0) AS revenue
FROM fact_sales fs JOIN dim_time dt ON fs.date_key=dt.date_key
GROUP BY dt.year, dt.month
ORDER BY dt.year, dt.month ASC
```

6.4 Agent 3 -- SQL Validation

Input: SQL string. Output: ValidationResult(is_valid, errors, warnings). Purely rule-based -- no LLM call. This agent enforces:

- Forbidden keyword check: regex blocks DELETE, DROP, UPDATE, INSERT, TRUNCATE, ALTER, EXEC, GRANT, CREATE.
- SELECT-only: query must begin with SELECT after stripping whitespace.
- sqlparse syntax: verifies the query parses without error.
- Table allowlist: any table not in {fact_sales, dim_customers, dim_products, dim_sellers, dim_time} is rejected.
- Complexity warnings: >5 JOINS or >2 subqueries trigger advisory warnings (non-blocking).

6.5 Agent 4 -- SQL Execution

Input: Validated SQL. Output: pd.DataFrame. Executed with tenacity retry (3 attempts, exponential backoff). A per-session timeout is applied via SET SESSION MAX_EXECUTION_TIME.

Execution agent with retry (execution_agent.py)

```
@retry(stop=stop_after_attempt(3),
       wait=wait_exponential(multiplier=1, min=1, max=4),
       reraise=True)
def execute_query(sql: str) -> pd.DataFrame:
    with engine.connect() as conn:
        conn.execute(text('SET SESSION MAX_EXECUTION_TIME=30000'))
        result = conn.execute(text(sql)) # text() prevents % format errors
        return pd.DataFrame(result.fetchall(),
                             columns=list(result.keys()))
```

6.6 Agent 5 -- Insight Generation

Input: Original question + SQL + top-20 rows of DataFrame (markdown table). Output: Dict with four keys: summary, key_finding, trend, recommendation. Acts as a senior e-commerce business analyst with temperature=0.3.

7. ReAct Agentic Loop & Reflection Agent

7.1 From Linear to Agentic

The original pipeline was a simple for-loop: Agent 1 ran once, raw error strings were passed back to Agent 2, and empty results were silently accepted. Four problems were identified: (1) wrong intent never corrected, (2) cryptic MySQL errors not actionable, (3) 0-row results silently accepted, (4) no reasoning between attempts.

7.2 ReAct Pattern

The upgraded pipeline implements Observe -> Reason -> Act each iteration. After any failure, the Reflection Agent classifies the error, diagnoses root cause, and returns a structured fix instruction to Agent 2 (or an intent revision request to Agent 1).

7.3 Reflection Agent -- 8 Error Categories

Error Type	Fix Strategy
group_by_violation	Use dt.year/dt.month (INT columns) in GROUP BY, never YEAR(col)
unknown_column	Replace hallucinated column name with schema-verified equivalent; revise intent
syntax_error	Rewrite from scratch; check commas, parentheses, JOIN syntax
table_not_found	Replace invalid table with one of the 5 known tables
query_timeout	Add WHERE order_status='delivered' filter; add LIMIT
empty_result	Relax date/status filters; broaden WHERE clause
validation_no_select	Output bare SQL only -- strip all prose and markdown
forbidden_keyword	Generate SELECT-only; never use DDL/DML keywords

[i] Design Decision

Known error types (7 of 8) use purely rule-based fix instructions -- no LLM call. This keeps retry latency low. Only genuinely unknown errors escalate to an LLM diagnosis call.

7.4 Progress Callbacks & Agent Trace

The pipeline accepts an on_progress callback fired after every significant action. Each event is a dict

{agent, action, message, ...}. Events are appended to PipelineResult.agent_trace (visible in the UI Agent Trace tab) and serialised to logs/query_log.jsonl for audit purposes.

8. Streamlit User Interface

8.1 Application Pages

File	Page	Purpose
app/Home.py	Home	Hero banner, live KPI strip, 5-agent accordion, tech stack, nav cards
pages/1_Query.py	Query	Full NL2SQL: voice, cache, pipeline, charts, PDF, benchmarking
pages/2_Dashboard.py	Dashboard	9-section pre-built KPI dashboard with RFM, forecast
pages/3_History.py	History	Persistent query log browser with re-run, CSV export
pages/4_Admin.py	Admin	DB status, schema explorer, agent diagnostics, log viewer

8.2 Design System (styles.py)

- Dark background: #080d1a (near-black navy) with subtle grid dot pattern
- Glassmorphism cards: backdrop-filter blur(12px) with translucent borders
- Typography: Inter (Google Fonts) at weights 300-800
- Page transitions: @keyframes with 0.85s cubic-bezier easing and scale effect (0.99->1.0)
- Accent colours: Blue (#3b82f6), Cyan (#06b6d4), Amber (#f59e0b), Green (#10b981)
- Plotly theme: consistent dark-glass chart layout via chart_layout(**overrides) helper

8.3 Smart Chart Selection (chart_builder.py)

A decision tree selects the optimal chart type automatically -- no static chart per query.

Condition	Chart Type
Column contains customer_state / seller_state	Brazil bubble map (scatter_geo)
Column name matches time hints (year/month/date/quarter)	Line + area chart
Distribution query AND <=8 rows	Donut chart
Avg label length >11 chars OR >12 rows	Horizontal bar chart
Short codes / ordinal values	Vertical bar chart
>=2 numeric columns, no category	Scatter plot

8.4 Dashboard: 9 Analytical Sections

- Monthly Revenue Trend -- line + dual-axis bar for order count
- Revenue by State -- vertical bar; Top Categories -- horizontal bar
- Payment Type Distribution -- donut; Average Review Score -- gauge
- Delivery Time Distribution -- histogram
- Seller Performance -- scatter (review score vs. revenue)
- Review Score Distribution + Best-Rated Categories
- Late Delivery Rate by State + Freight as % of Price
- Year-over-Year Quarterly Revenue (2017 vs. 2018) + Credit Card Instalment Analysis
- RFM Customer Intelligence -- KMeans segmentation (added as 10th feature upgrade)

9. Advanced Features (10 Upgrades)

Ten major features were added to demonstrate data science, ML, and product engineering depth. All were implemented in a single commit (897eabc) after the core system was stable.

Feature 1: Multi-Turn Conversational Memory

Module: app/components/conversation.py. Each successful query turn is stored in st.session_state. The last 4 turns are serialised as a context block and injected into Agent 1's prompt, enabling follow-up questions without repeating context.

Conversation context block injected into Agent 1

```
Previous conversation turns (for context):
Turn 1: 'Show revenue by state'
    -> metric=revenue, dimension=customer_state, result=27 rows
    -> key finding: Sao Paulo generates 43% of total revenue.
Turn 2: 'Now filter to just 2018'
    -> filters={year: 2018}, result=27 rows
Current question: 'Which of those are above average?'
```

Feature 2: Brazil Geospatial Bubble Map

Module: chart_builder.py. When a query result contains customer_state / seller_state, the chart builder automatically renders a Plotly scatter_geo map of Brazil. All 27 state centroids are hardcoded as (lat, lon) pairs -- no external GeoJSON file required. Bubble radius is proportional to the metric value.

Feature 3: Time-Series Forecasting

Module: analytics/forecasting.py. Any time-series result chart gains a 'Show 6-period forecast' toggle. Model: Ridge Regression with seasonality features [t, sin(2pit/12), cos(2pit/12)]. Confidence band = mean +/- RMSE of training residuals. Caption reports predicted next value, % change vs last actual, and training R^2.

Feature 4: PDF Report Export

Module: utils/report_generator.py. After a successful query, a Generate Report button produces a downloadable PDF using fpdf2. Contains: branded header, metadata strip (rows, duration, retries, provider, confidence), the query, chart image, full SQL, and all four AI insight cards.

Feature 5: FastAPI REST Endpoint

Method	Path	Description
GET	/health	Liveness check with live DB connectivity test

Method	Path	Description
GET	/schema	Star-schema table and column definitions
POST	/query	Full pipeline: intent -> SQL -> execute -> insights
POST	/query/sql-only	Dry-run: intent -> SQL + validation (no DB execution)

Feature 6: RFM Customer Segmentation

Module: analytics/rfm.py. KMeans (k=4) on StandardScaler-normalised [recency_inv, frequency, monetary]. Clusters ranked by composite score and assigned labels:

Segment	Description
Champions	Recent, frequent, high-spend. Best customers.
Loyal	Steady buyers. Reward to elevate to Champions.
At-Risk	Previously active but lapsing. Time-sensitive win-back.
Lost/Dormant	Long inactive, low value. Low-cost re-engagement only.

Feature 7: Query Confidence Scoring

Module: utils/confidence.py. 0-100 score: +30 first-attempt pass, -10 per retry, +20 non-empty result, +20 no Reflection triggered, +30 all 4 insight fields substantive. Displayed as colour-coded badge (Green >=85, Amber >=60, Red <60).

Feature 8: Semantic Query Cache

Module: utils/query_cache.py. TF-IDF cosine similarity cache (threshold 0.88, 100-entry LRU). On a cache hit, returns stored result instantly with a 'From cache' badge. New results stored to logs/query_cache.json. Vectoriser re-fit after every store. Warmed from disk once per session. TF-IDF chosen over sentence-transformers to avoid an 80MB model download on first run.

Feature 9: Voice Input

Inline JS in 1_Query.py via st.components.v1.html. Invokes the browser's Web Speech API. Start/Stop toggle, interim transcription displayed live, final result shown with a Copy button (navigator.clipboard.writeText). Zero backend dependencies; works natively in Chrome/Edge.

Feature 10: Multi-LLM Benchmarking Mode

Sidebar toggle in 1_Query.py. When active, the same query is submitted to both Ollama and Groq simultaneously using Python threading. Results displayed side-by-side: latency, SQL, sample rows,

insight summary. Answers concretely which LLM is better for NL2SQL on this schema.

Parallel provider execution

```
t_ollama = threading.Thread(target=_run_provider, args=('ollama', ollama_model))
t_groq   = threading.Thread(target=_run_provider, args=('groq',   groq_model))
t_ollama.start(); t_groq.start()
t_ollama.join();  t_groq.join() # parallel execution
```

10. Test Suite

10.1 Overview

54 passing pytest tests covering all five agents, validation logic, ETL cleaning, and DB connectivity. All external dependencies (LLM, MySQL) are mocked -- suite runs offline in under 2 seconds.

Class / Module	What is Tested	Tests
TestQueryAgent	Intent JSON parsing, dict return, JSONDecodeError	3
TestSQLAgent	SQL return type, markdown stripping, error feedback injection	3
TestValidationAgent	SELECT pass, DELETE/DROP/INSERT reject, unknown table, JOIN warn	8
TestExecutionAgent	DataFrame return, DB error propagation with tenacity retry	2
TestInsightAgent	Dict return, empty-DF path, all 4 keys present	3
TestPipeline	End-to-end success (mocked), validation failure triggers retry	2
test_cleaning.py	Dedup, datetime parse, typo rename, category merge	12
test_feature_eng	Delivery time, delay, is_late, approval time derivation	8
test_validation.py	Additional SQL safety edge cases	7
test_db.py	Engine singleton, connection test (mocked)	6

10.2 Key Patterns

LLM mocking: all chat() calls patched with unittest.mock.patch to return deterministic JSON. DB mocking: get_engine() patched with MagicMock whose connect().execute() returns a pre-configured result object. tenacity retry decorator is preserved -- retry behaviour is tested end-to-end.

11. Development History (15 Commits)

11.1 Conventional Commits

All commits follow the Conventional Commits specification: type(scope): description. Types used: feat, fix, chore, refactor, perf.

Commit	Type(Scope)	Summary
09f8835	chore: init	Project scaffolding, .gitignore, requirements.txt, empty module placeholders
27ebb7d	feat: add all	First functional commit: ETL, schema DDL, 5-agent pipeline, basic Streamlit UI, OpenAI LLM
62397d8	fix(etl): truncate	FK-safe truncate+append pattern; fixes FK violations on ETL re-runs
37715a8	refactor(agents): ollama	Migrated all LLM calls from OpenAI to Ollama (llama3); added llm_client.py abstraction
4fa7cff	perf(dw): indexes	Single-column -> composite indexes on fact_sales for analytical query performance
ef4446f	feat(agents): groq	Added Groq cloud API as second LLM provider; runtime set_provider() switching
1751b64	fix: 5 bugs	Fixed SQL injection vector, missing text() wrapper, dashboard crash, session state, loguru init
3d1f5e8	fix(app): dashboard	Dashboard crash fix; 54-test suite added; UI overhaul with glassmorphism + Inter typography
7f8345c	feat(ui): expandable	Pipeline steps as <details> accordion; collapsed sidebar; chart_layout() helper
318aac2	feat(ui): polished	4-colour insight cards; 4 new dashboard sections; image strips; tab spacing
b121f87	fix(pipeline): SQL	SQL extraction from LLM prose; day-of-week chart removed; transitions slowed to 0.85s
3248ed9	refactor(charts): adaptive	Complete chart_builder.py rewrite with 7-type decision tree; YoY quarterly added
b25ea0d	feat(agents): ReAct	Reflection Agent with 8-category classifier; full ReAct agentic loop; progress callbacks; agent trace
897eabc	feat(features): 10	All 10 major upgrades: memory, maps, forecast, PDF, API, RFM, confidence, cache, voice, benchmarking

11.2 Security Constraint

[i] Permanent Security Rule

.env (MySQL credentials + API keys) and imp_stuff.txt are permanently excluded from all commits via .gitignore. GitHub Push Protection blocked one attempted push where a real API key was accidentally included in .env.example -- corrected with a sanitised replacement commit.

12. Technical Challenges & Solutions

12.1 MySQL ONLY_FULL_GROUP_BY

Problem:

MySQL 8.0 enforces ONLY_FULL_GROUP_BY by default. LLMs frequently generate YEAR(column) in GROUP BY while referencing the raw column in SELECT -- valid in MySQL 5.7, rejected in 8.0 with Error 1055.

Solution:

Updated SQL agent prompt with explicit WRONG/CORRECT examples. Reflection Agent classifies this error immediately and returns: 'use dt.year and dt.month (INT columns) in GROUP BY, never YEAR(col)'.

12.2 LLM Prose Wrapping SQL

Problem:

Mistral and occasionally llama3 wrap generated SQL in explanation text. Old code only stripped markdown fences (````sql`), leaving prose intact -- causing Agent 3 'must begin with SELECT' failures.

Solution:

Two-step extraction: (1) find first SELECT keyword position, (2) split on first blank line after SELECT. Model-agnostic; handles any amount of prose before or after the SQL.

12.3 Day-of-Week Chart Always Blank

Problem:

Dashboard had a 'Revenue by Day of Week' chart mapping integer day codes to names. Investigation revealed dim_time.day_of_week is populated with string names ('Monday', 'Tuesday') via pandas .day_name(), not integers. The mapping produced all NaN.

Solution:

Removed the chart entirely (wrong architecture). Replaced with YoY quarterly revenue comparison chart -- more analytically valuable and correctly typed.

12.4 Duplicate Keyword Arguments in chart_layout()

Problem:

Plotly fig.update_layout(**CHART_LAYOUT, yaxis=...) raised TypeError: multiple values for keyword argument 'yaxis' because CHART_LAYOUT already contained a yaxis key.

Solution:

Implemented chart_layout(**overrides) helper: return {**CHART_LAYOUT, **overrides}. All callers updated to use this helper -- overrides win without conflict.

12.5 ETL Re-run FK Violations

Problem:

`to_sql(if_exists='replace')` drops and recreates dimension tables, but the fact table's foreign keys still reference old dimension table IDs.

Solution:

Disable FK checks, truncate all tables in fact-first order, re-enable FK checks, then re-insert. Makes ETL runs idempotent across multiple executions.

12.6 %Y/%m Format Errors in `pd.read_sql`

Problem:

Any SQL containing %Y or %m raises Python `TypeError` because PyMySQL interprets % as a Python format specifier.

Solution:

All SQL executed via SQLAlchemy must be wrapped with `sqlalchemy.text(sql)`. This treats the string as literal SQL, bypassing Python's format-string mechanism.

12.7 GitHub Push Protection

Problem:

A real OpenAI API key was accidentally included in `.env.example`. GitHub's Push Protection blocked the push.

Solution:

Corrected `.env.example` with placeholder values, amended the commit (safe since not yet pushed). Reinforced `.gitignore` rule for `.env` permanently.

13. Results & Evaluation

13.1 System Capabilities

Capability	Details
Natural-language queries	Any English business question about the Olist dataset
SQL generation	Valid MySQL 8.0 with correct GROUP BY, JOIN, aggregation
Self-correction	Up to 3 retry attempts with structured Reflection Agent guidance
Chart types	7 adaptive types including Brazil geo maps and dual-axis time series
Forecasting	6-period ahead with confidence band, R^2 , and % change caption
PDF export	Branded report with query, SQL, chart image, AI insights
Conversational memory	Last 4 turns; supports follow-up and refinement questions
REST API	4 endpoints with Swagger docs; provider + model per request
Semantic cache	TF-IDF cosine similarity; threshold 0.88; 100-entry LRU
Voice input	Web Speech API; Chrome/Edge; copy-to-clipboard
RFM segmentation	KMeans $k=4$; Champions/Loyal/At-Risk/Lost with revenue share
Multi-LLM benchmarking	Ollama + Groq parallel, side-by-side comparison
Confidence scoring	0-100 score with colour-coded badge per query
Query history	Persistent JSONL log; searchable; re-run support
Test coverage	54 passing tests; all agents and ETL functions covered

13.2 Performance

Scenario	Latency
Ollama local (llama3)	35-55 seconds end-to-end
Groq cloud (llama3-70b)	2-6 seconds end-to-end

Scenario	Latency
Cache hit	<100ms (TF-IDF cosine only)
ETL full load	~4 minutes (100K+ rows)
Dashboard first load	8-12 seconds; <1s from Streamlit cache

13.3 Sample SQL Generated

Example pipeline output

```
-- 'Monthly revenue trend for 2018'
SELECT dt.year, dt.month,
       CONCAT(dt.year, '-', LPAD(dt.month, 2, '0')) AS month_label,
       ROUND(SUM(fs.payment_value), 0) AS revenue,
       COUNT(DISTINCT fs.order_id) AS orders
FROM   fact_sales fs
JOIN   dim_time dt ON fs.date_key = dt.date_key
WHERE  fs.order_status = 'delivered' AND dt.year = 2018
GROUP BY dt.year, dt.month
ORDER BY dt.year, dt.month ASC
```

14. Conclusion & Future Work

14.1 What Was Built

- Kimball star-schema MySQL warehouse with 4 dimension tables, fact table, and composite indexes.
- Five-stage ETL pipeline with cleaning, feature engineering, and idempotent FK-safe reloading.
- Five-agent NL2SQL pipeline: structured intent parsing, schema-aware SQL generation, rule-based safety validation, retry-enabled execution, and LLM-generated business insights.
- ReAct agentic loop with Reflection Agent that classifies failures and plans targeted recovery.
- Polished four-page Streamlit UI with glassmorphism design, adaptive charts, and persistent query history.
- Ten advanced features: RFM, forecasting, PDF, REST API, cache, voice, benchmarking, memory, maps, confidence.
- 54 passing unit and integration tests with mocked LLM and DB dependencies.

14.2 Key Learnings

- Schema specificity is critical for NL2SQL: generic prompts fail; MySQL 8.0-specific WRONG/CORRECT examples dramatically reduce failures.
- Structured error feedback beats raw error passthrough: classified fix instructions reliably guide Agent 2 to the correct fix.
- Separation of concerns enables safe evolution: adding the Reflection Agent required zero changes to the 5 existing agents.
- ETL quality directly determines NL2SQL accuracy: schema typos and wrong column types propagate into broken SQL generations.
- Open-source LLMs are production-viable: Ollama + llama3 produces correct SQL for this schema in ~95% of first-attempt cases.

14.3 Future Work

- Fine-tune a small LLM (Phi-3 Mini) on the 20 benchmark queries with their ground-truth SQL.
- Schema-agnostic mode: auto-discover schema from any connected MySQL database and adapt prompts dynamically.
- Streaming pipeline: FastAPI WebSockets to stream agent progress events to the UI in real time.
- Churn prediction: train a customer churn classifier on RFM features; add Churn Risk column to the dashboard.
- Embedding-based cache: replace TF-IDF with sentence-transformers all-MiniLM-L6 for richer semantic matching.
- Docker: package the entire system in a docker-compose.yml for one-command deployment.

Appendix: Module Summary

File	Primary Responsibility
config.py	Reads .env via python-dotenv; exposes all config constants
run_etl.py	ETL entry point; calls all 5 stages in order
utils/db_connection.py	SQLAlchemy engine singleton; test_connection() health check
utils/llm_client.py	Unified Ollama/Groq dispatcher; runtime set_provider()
utils/logger.py	loguru setup; JSONL file sink at logs/query_log.jsonl
utils/confidence.py	0-100 pipeline confidence scorer
utils/query_cache.py	TF-IDF cosine semantic cache: warm, lookup, store
utils/report_generator.py	fpdf2 PDF report builder
etl/data_ingestion.py	Reads all 9 Olist CSVs into a dict of DataFrames
etl/data_cleaning.py	Per-table dedup, type coercion, string normalisation
etl/data_integration.py	Joins all tables into master order-level DataFrame
etl/feature_engineering.py	Derives delivery_time, delay, is_late, approval_hours
etl/data_loading.py	FK-safe truncate + writes dim and fact tables
agents/query_agent.py	Agent 1: NL -> JSON intent (with conversation history)
agents/sql_agent.py	Agent 2: intent -> MySQL SELECT (with error feedback)
agents/validation_agent.py	Agent 3: rule-based SQL safety + schema validation
agents/execution_agent.py	Agent 4: tenacity retry DB execution -> DataFrame
agents/insight_agent.py	Agent 5: LLM business insights from result DataFrame
agents/reflection_agent.py	Error classifier + recovery planner (8 categories)
agents/pipeline.py	ReAct orchestrator; progress callbacks; agent trace
analytics/forecasting.py	Ridge regression + seasonality time-series forecasting

File	Primary Responsibility
analytics/rfm.py	RFM KMeans segmentation; segment labels and revenue share
api/main.py	FastAPI: /health, /schema, /query, /query/sql-only
app/Home.py	Landing page: KPIs, 5-agent accordion, nav cards
app/components/styles.py	Global CSS injection; CHART_LAYOUT; COLOR_SEQ
app/components/chart_builder.py	7-type adaptive chart selection and rendering
app/components/conversation.py	Multi-turn history: add, clear, render, build context
app/components/insight_display.py	4-card AI insight renderer
app/pages/1_Query.py	Query page: voice, cache, pipeline, charts, PDF, benchmarking
app/pages/2_Dashboard.py	9-section dashboard: trends, RFM, forecasting
app/pages/3_History.py	JSONL log browser with search, filter, re-run, CSV export
app/pages/4_Admin.py	DB status, schema explorer, diagnostics, log viewer